

Logistic Regression, K-Nearest Neighbor, Naïve Bayes, Decision Tree

disusun untuk memenuhi
tugas Pembelajaran Mesin

oleh :

Kelompok 4

Glenn Hakim	2208107010072
Ahmad Syah Ramadhan	2208107010033
Andika Pebriansyah	2208107010058
Nisa Rianti	2208107010018
Nuri Masyithah	2208107010006



JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
2025

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan dunia kerja yang dinamis di era modern menuntut individu dan organisasi untuk mampu menyesuaikan diri dengan perubahan kebutuhan pasar. Salah satu aspek penting dalam pengelolaan sumber daya manusia adalah penempatan karir yang tepat, karena berdampak langsung pada produktivitas, kepuasan kerja, serta keberhasilan organisasi secara keseluruhan (Umar et al., 2018; Danuri & Jaroji, 2019; Metalita et al., 2021). Hal ini mendorong munculnya pendekatan berbasis data seperti analitik SDM, yang memungkinkan proses pengambilan keputusan dilakukan secara lebih akurat dan objektif.

Seiring berkembangnya teknologi, *machine learning* menjadi solusi yang banyak digunakan untuk membantu pengambilan keputusan yang kompleks, termasuk dalam penempatan karir. Dengan memanfaatkan data seperti riwayat pendidikan, pengalaman kerja, keterampilan, dan minat, model klasifikasi berbasis *machine learning* dapat merekomendasikan posisi kerja yang sesuai dengan karakteristik individu (Bao et al., 2023). Pendekatan ini dinilai efektif dalam meningkatkan efisiensi dan ketepatan dalam penempatan tenaga kerja.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

1. Bagaimana penerapan algoritma klasifikasi seperti Logistic Regression, K-Nearest Neighbor, Naïve Bayes, dan Decision Tree dalam membantu proses penempatan karir?
2. Algoritma mana yang memberikan hasil paling optimal dalam merekomendasikan posisi kerja berdasarkan data individu?

1.3 Tujuan

Tujuan dari analissi ini adalah:

1. Menganalisis dan mengimplementasikan beberapa algoritma machine learning untuk klasifikasi penempatan karir.
2. Membandingkan performa model Logistic Regression, K-Nearest Neighbor, Naïve Bayes, dan Decision Tree dalam memberikan rekomendasi posisi kerja berdasarkan atribut seperti pendidikan, pengalaman, dan keterampilan individu.

BAB II

PEMBAHASAN

2.1 Pemahaman Dataset

Dataset yang digunakan dalam analisis ini adalah *Job Placement Dataset.csv*, dataset ini bersumber dari *kaggle.com* dimana jumlah total data sebanyak 215 dan terdiri dari 12 atribut label dan 1 atribut target. Secara detail atribut atribut yang digunakan pada penelitian ini dijelaskan pada tabel 2.1.

Tabel 2.1 Atribut Dataset

Atribut	Keterangan
gender	Jenis kelamin kandidat
ssc_percentage	Persentase ujian sekolah menengah atas (Kelas 10)
ssc_board	Dewan pendidikan untuk ujian ssc
hsc_percentage	Persentase ujian sekolah menengah atas (Kelas 12)
hsc_board	Dewan pendidikan untuk ujian hsc
hsc_subject	Mata pelajaran untuk hsc
degree_percentage	Persentase nilai dalam gelar sarjana
undergrad_degree	Jurusan gelar sarjana
work_experience	Pengalaman kerja sebelumnya
emp_test_percentage	Persentase tes bakat
spesialisasi	Jurusan pasca sarjana - (spesialisasi MBA)
mba_percent	Persentase nilai dalam gelar MBA
status (target)	Ditempatkan / Tidak Ditempatkan
Sumber: (Raza, 2023)	

Pada Tabel 2.1 menjelaskan masing-masing keterangan dari setiap atribut penelitian dimana berdasarkan dari 12 atribut inputan akan menghasilkan klasifikasi atau target apakah kandidat ditempatkan atau tidak.

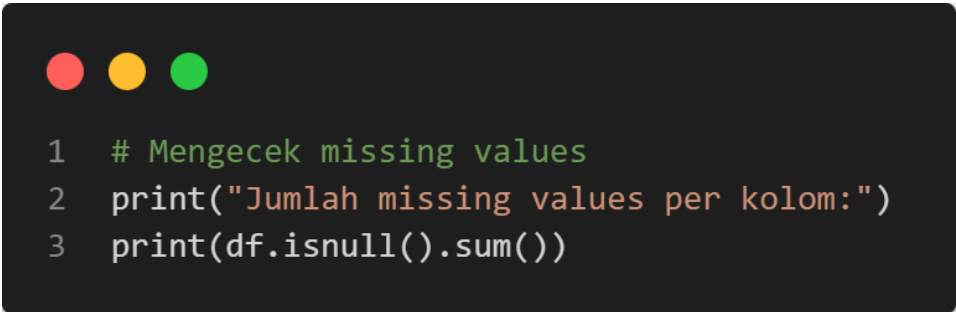
Statistik deskriptif dan visualisasi awal dilakukan untuk memahami distribusi data.

Tabel 2.2 Statistik deskriptif dan visualisasi awal

	ssc_percentage	hsc_percentage	degree_percentage	emp_test_percentage	mba_percent
count	215.000000	215.000000	215.000000	215.000000	215.000000
mean	67.303395	66.333163	66.370186	72.100558	62.278186
std	10.827205	10.897509	7.358743	13.275956	5.833385
min	40.890000	37.000000	50.000000	50.000000	51.210000
25%	60.600000	60.900000	61.000000	60.000000	57.945000
50%	67.000000	65.000000	66.000000	71.000000	62.000000
75%	75.700000	73.000000	72.000000	83.500000	66.255000
max	89.400000	97.700000	91.000000	98.000000	77.890000

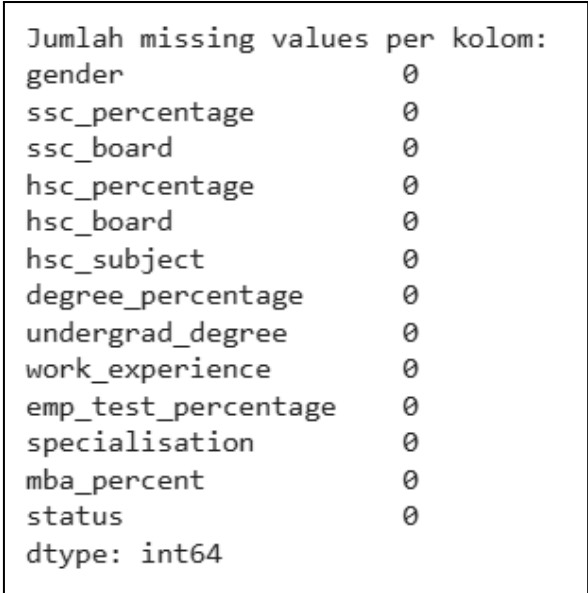
Gambar diatas menampilkan ringkasan statistik data numerik dari lima kolom persentase, yaitu nilai ujian SSC, HSC, gelar sarjana, tes kemampuan kerja, dan MBA. Setiap kolom memiliki 215 data tanpa nilai kosong. Rata-rata nilai berkisar antara 62–72%, dengan standar deviasi tertinggi pada emp_test_percentage (13.27) yang menunjukkan variasi nilai terbesar. Nilai minimum dan maksimum di masing-masing kolom menunjukkan rentang performa kandidat, dan distribusi nilai cukup seimbang ditandai dengan median yang mendekati rata-rata.

- Mengecek *missing values*



```
1 # Mengecek missing values
2 print("Jumlah missing values per kolom:")
3 print(df.isnull().sum())
```

Gambar 2.1 Kode Mengecek Missing Values\



```
Jumlah missing values per kolom:
gender                0
ssc_percentage        0
ssc_board             0
hsc_percentage        0
hsc_board             0
hsc_subject           0
degree_percentage     0
undergrad_degree      0
work_experience        0
emp_test_percentage   0
specialisation        0
mba_percent           0
status                0
dtype: int64
```

Gambar 2.2 Output Mengecek Missing Values

Pada gambar diatas, tidak terdapat Missing Values pada dataset.

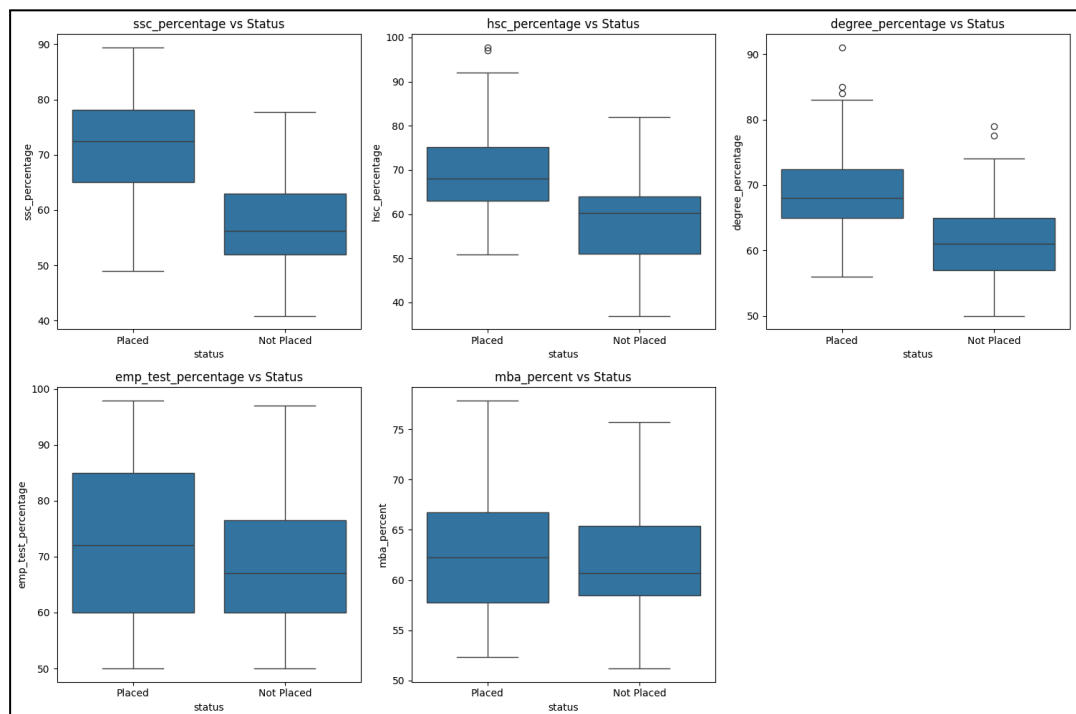
- **Distribusi persentase nilai berdasarkan status penempatan**

```

1 # Distribusi persentase nilai berdasarkan status penempatan
2 numerical_cols = ['ssc_percentage', 'hsc_percentage', 'degree_percentage',
3                  'emp_test_percentage', 'mba_percent']
4
5 plt.figure(figsize=(15, 10))
6 for i, col in enumerate(numerical_cols):
7     plt.subplot(2, 3, i+1)
8     sns.boxplot(x='status', y=col, data=df)
9     plt.title(f'{col} vs Status')
10 plt.tight_layout()

```

Gambar 2.3 Kode Distribusi Persentase Nilai Berdasarkan Status Penempatan



Gambar 2.4 Visualisasi distribusi persentase nilai berdasarkan status penempatan

Gambar di atas memiliki tiga nilai akademik awal—ssc_percentage, hsc_percentage, dan degree_percentage—memiliki perbedaan distribusi yang jelas antara mahasiswa yang Placed dan Not Placed, menunjukkan bahwa performa akademik dasar berpengaruh besar terhadap peluang penempatan kerja. Sebaliknya, distribusi emp_test_percentage dan mba_percent tampak mirip, mengindikasikan bahwa nilai tes kerja dan MBA tidak terlalu menentukan status penempatan.

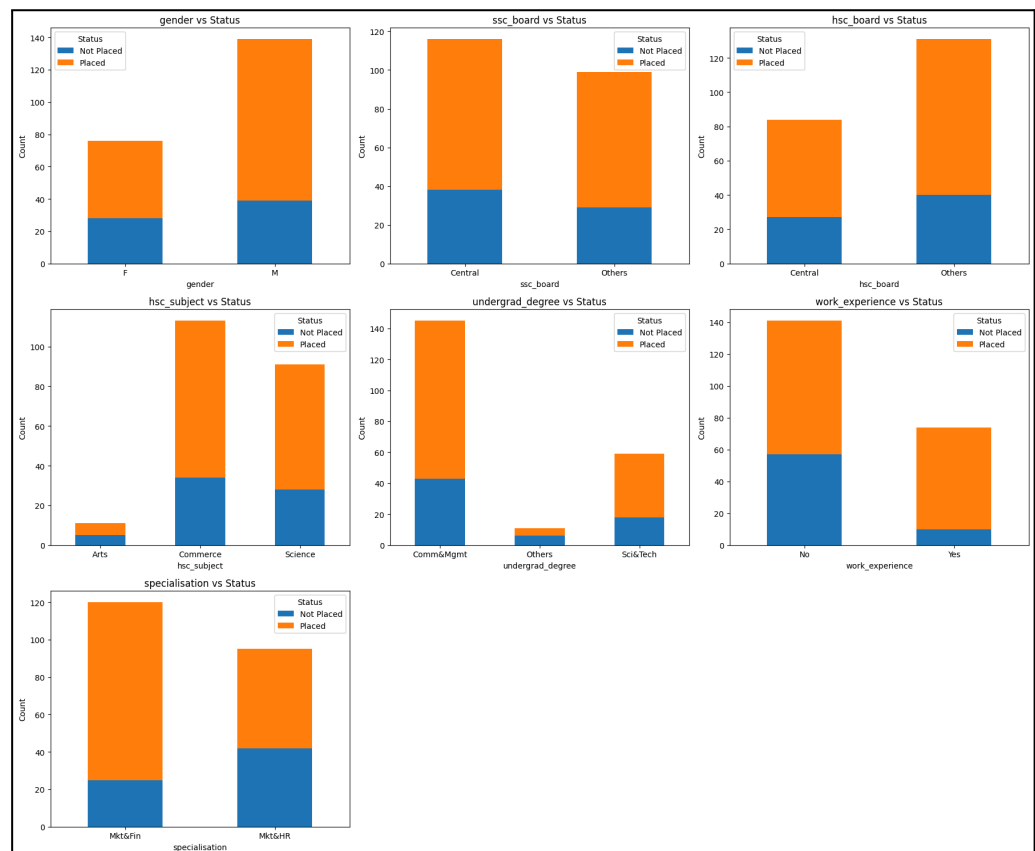
- Distribusi variabel kategorikal

```

1 # Distribusi variabel kategorikal
2 categorical_cols = ['gender', 'ssc_board', 'hsc_board', 'hsc_subject',
3                     'undergrad_degree', 'work_experience', 'specialisation']
4
5 fig, axes = plt.subplots(3, 3, figsize=(18, 15))
6 axes = axes.flatten()
7
8 for i, col in enumerate(categorical_cols):
9     if i < len(axes):
10         # Hitung jumlah untuk setiap kategori dan status
11         crosstab = pd.crosstab(df[col], df['status'])
12         crosstab.plot(kind='bar', stacked=True, ax=axes[i], rot=0)
13         axes[i].set_title(f'{col} vs Status')
14         axes[i].set_ylabel('Count')
15         axes[i].legend(title='Status')
16
17 # Hilangkan subplot yang tidak terpakai
18 for j in range(i+1, len(axes)):
19     fig.delaxes(axes[j])
20
21 plt.tight_layout()
22 plt.show()

```

Gambar 2.5 Kode Distribusi Variabel Kategorikal



Gambar 2.6 Visualisasi Distribusi Variabel Kategorikal

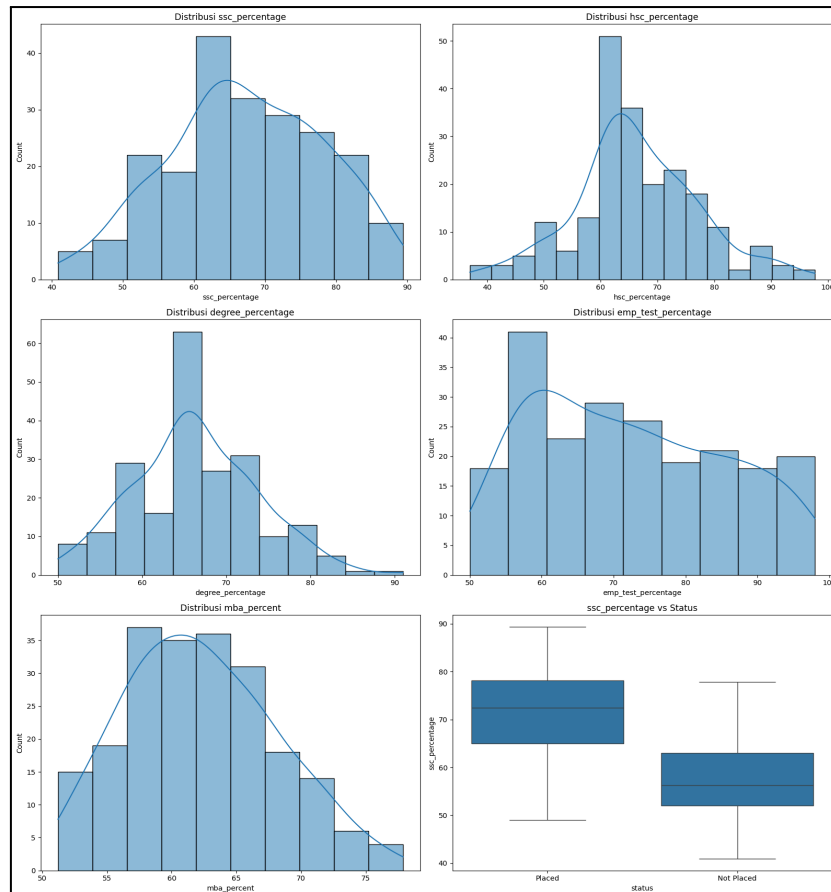
Berdasarkan visualisasi distribusi variabel kategorikal terhadap status penempatan kerja, dapat disimpulkan bahwa faktor-faktor seperti pengalaman kerja, latar belakang pendidikan di bidang Science & Technology, serta spesialisasi Marketing & HR memiliki pengaruh positif terhadap peluang penempatan kerja. Meskipun jumlah peserta dengan latar belakang tersebut lebih sedikit, proporsi yang berhasil ditempatkan cenderung lebih tinggi dibanding kelompok lainnya.

Sementara itu, variabel seperti gender, asal sekolah (SSC dan HSC board), serta jurusan di sekolah menengah tidak menunjukkan pengaruh signifikan terhadap status penempatan. Temuan ini mengindikasikan bahwa aspek keahlian teknis, pengalaman praktis, dan fokus spesialisasi berperan penting dalam keberhasilan penempatan kerja.

- **Analisis distribusi variabel numerik**

```
1 # Analisis distribusi variabel numerik
2 numerical_cols = ['ssc_percentage', 'hsc_percentage', 'degree_percentage',
3                  'emp_test_percentage', 'mba_percent']
4
5 fig, axes = plt.subplots(3, 2, figsize=(16, 18))
6 axes = axes.flatten()
7
8 for i, col in enumerate(numerical_cols):
9     if i < len(axes):
10         # Histogram untuk setiap variabel numerik
11         sns.histplot(data=df, x=col, kde=True, ax=axes[i])
12         axes[i].set_title(f'Distribusi {col}')
13
14         # Boxplot berdasarkan status
15         if i + len(numerical_cols) < len(axes):
16             sns.boxplot(x='status', y=col, data=df, ax=axes[i + len(numerical_cols)])
17             axes[i + len(numerical_cols)].set_title(f'{col} vs Status')
18
19 # Hilangkan subplot yang tidak terpakai
20 for j in range(i+1 + len(numerical_cols), len(axes)):
21     fig.delaxes(axes[j])
22
23 plt.tight_layout()
24 plt.show()
```

Gambar 2.7 Kode Analisis Distribusi Variabel Numerik



Gambar 2.8 Visualisasi Analisis Distribusi Variabel Numerik

Gambar di atas menunjukkan distribusi lima variabel numerik: nilai SSC, HSC, sarjana, tes kerja, dan MBA. Mayoritas nilai berada di kisaran menengah, dengan distribusi yang sebagian besar mendekati normal. Selain itu, boxplot pada nilai SSC menunjukkan bahwa peserta yang **Placed** umumnya memiliki nilai lebih tinggi dibandingkan yang **Not Placed**, yang mengindikasikan adanya hubungan positif antara performa akademik dan peluang penempatan kerja.

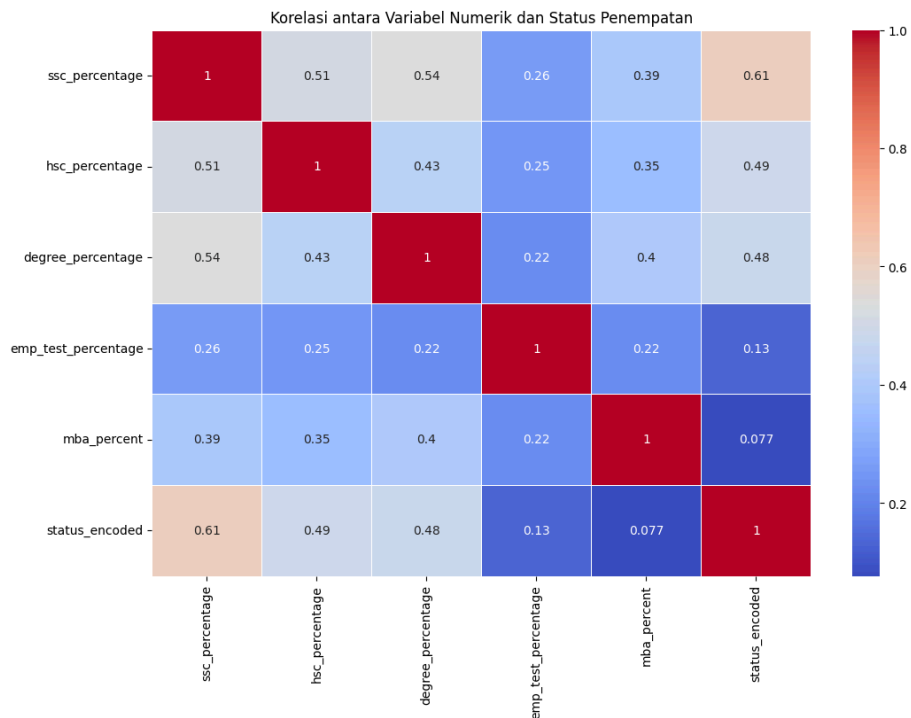
- **Korelasi persentase nilai dengan status penempatan**

```

1 # Korelasi persentase nilai dengan status penempatan
2 df_numeric = df.copy()
3 df_numeric['status_encoded'] = df_numeric['status'].map({'Placed': 1, 'Not Placed': 0})
4
5 correlation = df_numeric[numerical_cols + ['status_encoded']].corr()
6 plt.figure(figsize=(12, 8))
7 sns.heatmap(correlation, annot=True, cmap='coolwarm', linewidths=0.5)
8 plt.title('Korelasi antara Variabel Numerik dan Status Penempatan')
9 plt.show()

```

Gambar 2.8 Korelasi Persentase Nilai dengan Status Penempatan



Gambar 2.8 Visualisasi Korelasi Persentase Nilai dengan Status Penempatan

Berdasarkan hasil analisis korelasi yang ditampilkan pada heatmap di atas, diketahui bahwa variabel `ssc_percentage` memiliki hubungan paling kuat dengan status penempatan kerja (`status_encoded`), dengan nilai korelasi sebesar 0.61. Ini menunjukkan bahwa semakin tinggi nilai SSC seseorang, maka kemungkinan untuk ditempatkan kerja juga semakin besar.

Selain itu, variabel `hsc_percentage` dan `degree_percentage` juga menunjukkan korelasi sedang terhadap status penempatan, masing-masing sebesar 0.49 dan 0.48. Sementara itu, `emp_test_percentage` dan `mba_percent` memiliki korelasi yang lebih rendah, yaitu hanya 0.13 dan 0.077, sehingga dapat disimpulkan bahwa nilai akademik dasar memiliki pengaruh yang lebih besar terhadap keberhasilan penempatan kerja dibandingkan dengan nilai tes kerja maupun nilai MBA.

2.2 Eksplorasi Data dan Pra-pemrosesan

- Membuat fitur baru dari fitur yang ada

```

1 # Membuat fitur baru dari fitur yang ada
2 df['ssc_hsc_diff'] = df['hsc_percentage'] - df['ssc_percentage'] # Selisih nilai HSC dan SSC
3 df['degree_mba_diff'] = df['mba_percent'] - df['degree_percentage'] # Selisih nilai MBA dan Degree
4 df['overall_academic_avg'] = (df['ssc_percentage'] + df['hsc_percentage'] + df['degree_percentage']) / 3 # Rata-rata akademik
5 df['academic_emp_ratio'] = df['overall_academic_avg'] / df['emp_test_percentage'] # Rasio nilai akademik dan employment test

```

Gambar 2.9 Membuat fitur baru dari fitur yang ada

Pada tahap eksplorasi dan pra-pemrosesan data, dilakukan pembuatan beberapa fitur baru untuk memperkaya informasi dari data asli. Fitur-fitur tersebut meliputi `ssc_hsc_diff` (selisih antara nilai HSC dan SSC), `degree_mba_diff` (selisih nilai MBA dan sarjana), `overall_academic_avg` (rata-rata nilai akademik dari SSC, HSC, dan sarjana), serta `academic_emp_ratio` (rasio antara rata-rata nilai akademik dengan nilai tes kerja). Fitur ini dibuat untuk membantu mengidentifikasi pola perkembangan akademik peserta dan hubungannya dengan kemampuan kerja.

	gender	ssc_percentage	ssc_board	hsc_percentage	hsc_board	hsc_subject	degree_percentage	undergrad_degree	work_experience	emp_test_percentage	specialisation	mba_percent	status	ssc_hsc_diff	degree_mba_diff	overall_academic_avg	academic_emp_ratio
0	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0	MM&HR	58.80	Placed	24.0	0.80	72.000000	1.309091
1	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5	MM&Fin	66.28	Placed	-1.0	-11.20	78.380000	0.906127
2	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0	MM&Fin	57.80	Placed	3.0	-6.20	65.666667	0.875556
3	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0	MM&HR	59.43	Not Placed	-4.0	7.43	53.333333	0.808081
4	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8	MM&Fin	55.50	Placed	-12.2	-17.80	77.566667	0.801309

Gambar 2.10 Data Setelah Pembuatan Fitur Baru

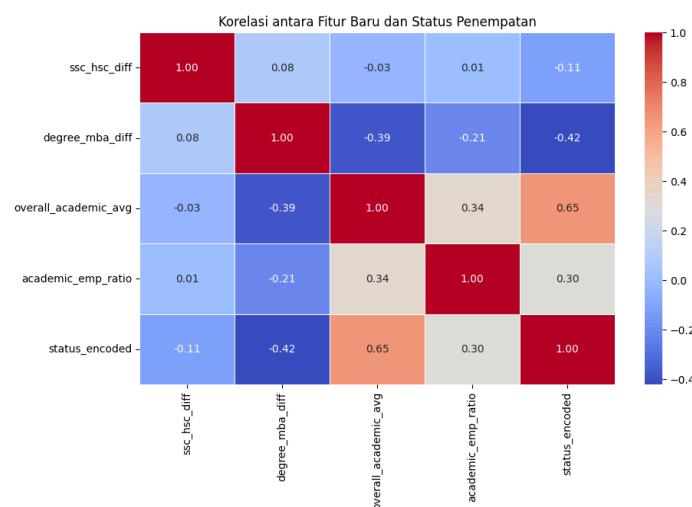
- Cek korelasi fitur baru dengan status

```

1 # Cek korelasi fitur baru dengan status
2 df_numeric = df.copy()
3 df_numeric['status_encoded'] = df_numeric['status'].map({'Placed': 1, 'Not Placed': 0})
4 new_numerical_cols = ['ssc_hsc_diff', 'degree_mba_diff', 'overall_academic_avg', 'academic_emp_ratio']
5
6 correlation_new = df_numeric[new_numerical_cols + ['status_encoded']].corr()
7 plt.figure(figsize=(10, 6))
8 sns.heatmap(correlation_new, annot=True, cmap='coolwarm', linewidths=0.5, fmt='.2f')
9 plt.title('Korelasi antara Fitur Baru dan Status Penempatan')
10 plt.show()

```

Gambar 2.11 Kode Cek korelasi fitur baru dengan status



Gambar 2.12 Visualisasi korelasi fitur baru dengan status

Gambar di atas menunjukkan sebuah heatmap korelasi yang menggambarkan hubungan antar fitur baru yang dihasilkan dengan status penempatan kerja (diwakili oleh `status_encoded`). Dari visualisasi tersebut, dapat dilihat bahwa fitur `overall_academic_avg` memiliki korelasi paling kuat dan positif terhadap status penempatan, dengan nilai korelasi sebesar 0.65. Artinya, semakin tinggi rata-rata nilai akademik seseorang, maka semakin besar kemungkinan mereka untuk mendapatkan pekerjaan. Selain itu, fitur `academic_emp_ratio` juga menunjukkan korelasi positif sedang (0.30), yang berarti bahwa rasio antara performa akademik dan pengalaman kerja juga memiliki kontribusi terhadap status penempatan.

Sebaliknya, fitur `degree_mba_diff` menunjukkan korelasi negatif cukup kuat terhadap status penempatan dengan nilai -0.42. Hal ini mengindikasikan bahwa semakin besar selisih antara nilai gelar sarjana dan nilai MBA, maka semakin kecil kemungkinan seseorang untuk mendapatkan pekerjaan, yang mungkin disebabkan oleh ketidakkonsistenan performa akademik. Fitur lainnya seperti `ssc_hsc_diff` dan `academic_emp_ratio` memiliki korelasi yang lemah terhadap status penempatan, menunjukkan bahwa pengaruhnya tidak terlalu signifikan. Secara keseluruhan, heatmap ini membantu mengidentifikasi fitur mana yang paling relevan dan potensial dalam membangun model prediksi status penempatan kerja.

- **Encoding variabel kategorikal**

```
1 # Encoding variabel kategorikal
2 categorical_cols = ['gender', 'ssc_board', 'hsc_board', 'hsc_subject',
3                     'undergrad_degree', 'work_experience', 'specialisation']
4
5 # Gunakan One-Hot Encoding untuk fitur kategorikal
6 df_encoded = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True)
7
8 # Encode target variable
9 df_encoded['status'] = df_encoded['status'].map({'Placed': 1, 'Not Placed': 0})
10
11 # Menampilkan 5 baris pertama data setelah encoding
12 df_encoded.head()
```

Gambar 2.13 Kode Encoding variabel kategorikal

Kode tersebut melakukan proses encoding untuk mengubah variabel kategorikal menjadi bentuk numerik agar dapat digunakan dalam model

machine learning. Kolom-kolom seperti jenis kelamin, jurusan, dan pengalaman kerja diubah menggunakan One-Hot Encoding, sementara kolom target status dikonversi menjadi 1 untuk 'Placed' dan 0 untuk 'Not Placed'. Hasilnya adalah data yang sepenuhnya numerik dan siap untuk dianalisis atau dilatih dalam model prediksi.

	ssc_percentage	hsc_percentage	degree_percentage	emp_test_percentage	mba_percent	status	ssc_hsc_diff	degree_mba_diff	overall_academic_avg	academic_emp_ratio	gender_F	ssc_board_others	hsc_board_others	hsc_subject_Commerce	hsc_subject_Science	undergrad_degree_others	undergrad_degree_SciTech	work_experience_Yes	specialisation_Marketing
0	87.00	91.00	88.00	85.0	88.80	1	24.0	0.80	72.333333	1.500001	True	True	True	True	False	False	True	False	True
1	79.33	78.33	77.48	88.0	88.28	1	-1.0	-11.20	78.380000	0.809127	True	False	True	False	True	False	True	True	False
2	86.00	86.00	84.00	75.0	87.80	1	3.0	-8.20	85.966667	0.878556	True	False	False	False	False	False	False	False	False
3	86.00	82.00	82.00	88.0	88.43	0	-4.0	7.43	83.333333	0.800081	True	False	False	False	True	False	True	False	True
4	85.80	73.80	73.30	90.8	85.50	1	-12.2	-17.80	77.500007	0.801309	True	False	False	True	False	False	False	False	False

Gambar 2.14 *Data Setelah Encoding*

- **Standardization untuk fitur numerik**

```

1 # Standardization untuk fitur numerik
2 numerical_cols = ['ssc_percentage', 'hsc_percentage', 'degree_percentage',
3                   'emp_test_percentage', 'mba_percent', 'ssc_hsc_diff',
4                   'degree_mba_diff', 'overall_academic_avg', 'academic_emp_ratio']
5
6 scaler = StandardScaler()
7 df_encoded[numerical_cols] = scaler.fit_transform(df_encoded[numerical_cols])
8
9 # Menampilkan 5 baris pertama data setelah standardization
10 df_encoded.head()

```

Gambar 2.15 *Kode Standardization untuk Fitur Numerik*

Kode tersebut melakukan standardisasi pada fitur-fitur numerik agar memiliki skala yang seragam. Standardisasi ini dilakukan menggunakan `StandardScaler()` dari pustaka `sklearn`, yang mengubah nilai setiap fitur menjadi distribusi dengan rata-rata 0 dan standar deviasi 1. Kolom-kolom numerik yang distandarisasi antara lain nilai-nilai akademik seperti `ssc_percentage`, `hsc_percentage`, hingga fitur turunan seperti `ssc_hsc_diff` dan `overall_academic_avg`. Langkah ini penting agar fitur-fitur tersebut tidak memiliki skala yang berbeda-beda, sehingga model machine learning tidak bias terhadap fitur dengan nilai yang lebih besar. Setelah proses ini, data ditampilkan kembali untuk memastikan hasilnya sudah dalam skala yang distandarisasi.

	sci_percentage	hsc_percentage	degree_percentage	emp_tech_percentage	mba_percent	status	sci_hsc_diff	degree_mba_diff	overall_science_avg	science_emp_ratio	gender_F	sci_board_Others	hsc_board_Others	sci_subject_Commerce	sci_subject_Science	undergrad_degree_Others	undergrad_degree_SciTech	work_experience_Yes	specialization_MBAHR
0	-0.020087	2.208912	-1.140102	-1.291091	-0.597947	1	2.331010	-0.858034	0.073507	1.900003	True	True	True	True	False	True	True	False	True
1	1.113389	1.103440	1.513287	1.087187	0.887620	1	-0.002779	-0.872823	1.478800	-0.248537	True	False	True	False	True	False	True	True	False
2	-0.213238	0.153313	-0.322843	0.218908	-0.789474	1	0.370527	-0.238507	-0.120548	-0.416254	True	False	False	False	False	False	False	False	False
3	-1.048417	-1.318339	-1.807382	-0.400592	-0.480395	0	-0.262833	1.576937	-1.055138	-0.700830	True	False	False	False	True	False	True	False	True
4	1.712332	0.886391	0.943958	1.864808	-1.194876	1	-1.046318	-1.878120	1.377084	-0.828434	True	False	False	True	False	False	False	False	False

Gambar 2.16 Data Setelah Standardization

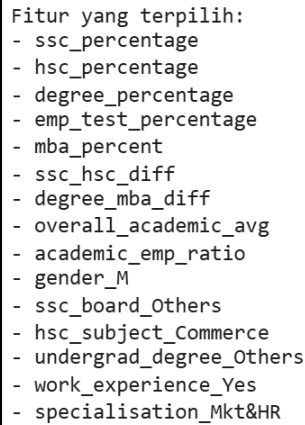
- **Persiapan data untuk pemodelan**

```

1 # Mempersiapkan data untuk pemodelan
2 X = df_encoded.drop('status', axis=1)
3 y = df_encoded['status']
4
5 # Pilih fitur yang paling berpengaruh menggunakan SelectKBest
6 selector = SelectKBest(f_classif, k=15)
7 X_selected = selector.fit_transform(X, y)
8
9 # Dapatkan indeks fitur yang terpilih
10 selected_indices = selector.get_support(indices=True)
11 selected_features = X.columns[selected_indices]
12
13 print("Fitur yang terpilih:")
14 for feature in selected_features:
15     print(f"- {feature}")
16
17 # Gunakan hanya fitur yang terpilih
18 X = X[selected_features]
```

Gambar 2.17 Kode Standardization untuk Fitur Numerik

Kode ini digunakan untuk mempersiapkan data sebelum dimasukkan ke dalam model machine learning. Pertama, data dibagi menjadi fitur (X) dan target (y), di mana kolom status menjadi target. Selanjutnya, digunakan teknik seleksi fitur SelectKBest dengan metode f_classif untuk memilih 15 fitur terbaik yang paling berpengaruh terhadap target. Setelah fitur terbaik terpilih, hanya fitur-fitur tersebut yang digunakan dalam variabel X untuk proses pemodelan. Langkah ini membantu meningkatkan akurasi model dan mengurangi kompleksitas data.

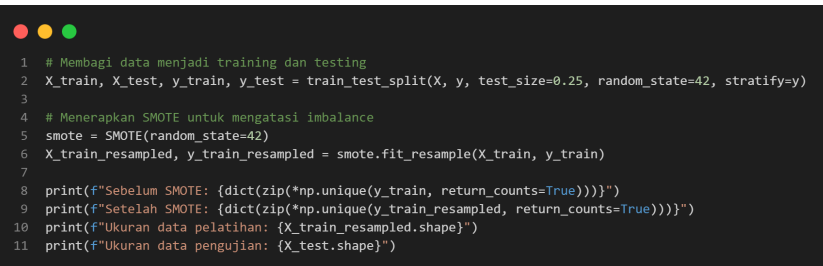


Fitur yang terpilih:

- ssc_percentage
- hsc_percentage
- degree_percentage
- emp_test_percentage
- mba_percent
- ssc_hsc_diff
- degree_mba_diff
- overall_academic_avg
- academic_emp_ratio
- gender_M
- ssc_board_Others
- hsc_subject_Commerce
- undergrad_degree_Others
- work_experience_Yes
- specialisation_Mkt&HR

Gambar 2.18 *Fitur Terpilih*

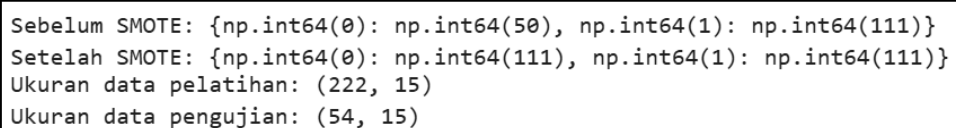
- **Membagi data menjadi training dan testing**



```
1 # Membagi data menjadi training dan testing
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
3
4 # Menerapkan SMOTE untuk mengatasi imbalance
5 smote = SMOTE(random_state=42)
6 X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
7
8 print(f"Sebelum SMOTE: {dict(zip(*np.unique(y_train, return_counts=True)))}")
9 print(f"Setelah SMOTE: {dict(zip(*np.unique(y_train_resampled, return_counts=True)))}")
10 print(f"Ukuran data pelatihan: {X_train_resampled.shape}")
11 print(f"Ukuran data pengujian: {X_test.shape}")
```

Gambar 2.19 *Kode Membagi data menjadi training dan testing*

Potongan kode ini membagi data menjadi data pelatihan (training) dan data pengujian (testing) dengan proporsi 75:25 menggunakan `train_test_split`. Parameter `stratify=y` memastikan bahwa proporsi kelas target tetap seimbang di kedua set. Karena data target tidak seimbang (jumlah 'Placed' dan 'Not Placed' berbeda), diterapkan SMOTE (Synthetic Minority Over-sampling Technique) untuk menyeimbangkan data pelatihan dengan menambahkan contoh sintetis dari kelas minoritas. Setelah SMOTE, jumlah data untuk kedua kelas menjadi seimbang. Langkah ini penting agar model tidak bias terhadap kelas mayoritas.



```
Sebelum SMOTE: {np.int64(0): np.int64(50), np.int64(1): np.int64(111)}
Setelah SMOTE: {np.int64(0): np.int64(111), np.int64(1): np.int64(111)}
Ukuran data pelatihan: (222, 15)
Ukuran data pengujian: (54, 15)
```

Gambar 2.20 *Hasil Membagi data menjadi training dan testing*

2.3 Implementasi Model

- Fungsi untuk evaluasi model

```
1 # Fungsi untuk evaluasi model
2 def evaluate_model(model, X_train, y_train, X_test, y_test, model_name):
3     # Cross Validation pada data training
4     cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
5     cv_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
6
7     # Fit model dengan semua data training
8     model.fit(X_train, y_train)
9
10    # Prediksi
11    y_pred = model.predict(X_test)
12    y_proba = model.predict_proba(X_test)[:, 1]
13
14    # Hitung metrik evaluasi
15    accuracy = accuracy_score(y_test, y_pred)
16    conf_matrix = confusion_matrix(y_test, y_pred)
17    class_report = classification_report(y_test, y_pred)
18    loss = log_loss(y_test, y_proba)
19
20    # ROC Curve
21    fpr, tpr, _ = roc_curve(y_test, y_proba)
22    roc_auc = auc(fpr, tpr)
23
24    # Output hasil
25    print(f"\n{'-'*50}\n{model_name}:")
26    print(f"Cross-Validation Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
27    print(f"Test Accuracy: {accuracy:.4f}")
28    print(f"ROC AUC: {roc_auc:.4f}")
29    print(f"Log Loss: {loss:.4f}")
30    print("\nConfusion Matrix:")
31    print(conf_matrix)
32    print("\nClassification Report:")
33    print(class_report)
34
35    return {
36        'model': model,
37        'name': model_name,
38        'accuracy': accuracy,
39        'cv_accuracy': cv_scores.mean(),
40        'roc_auc': roc_auc,
41        'log_loss': loss,
42        'fpr': fpr,
43        'tpr': tpr,
44        'y_pred': y_pred,
45        'y_proba': y_proba
46    }
```

Gambar 2.21 Kode Fungsi Evaluasi Model

Fungsi `evaluate_model` ini digunakan untuk mengevaluasi performa model machine learning secara menyeluruh. Pertama, dilakukan cross-validation sebanyak 5 kali pada data pelatihan untuk melihat kestabilan akurasi model. Kemudian model dilatih dengan seluruh data pelatihan dan diuji pada data pengujian. Hasil prediksi dibandingkan dengan data aktual untuk menghitung berbagai metrik evaluasi seperti akurasi, confusion matrix, classification report, log loss, dan ROC AUC. Fungsi ini juga menghasilkan data untuk plot kurva ROC. Semua hasil evaluasi ditampilkan secara lengkap dan dikembalikan dalam bentuk dictionary untuk dianalisis lebih lanjut. Ini membantu memilih model terbaik berdasarkan performa di berbagai aspek.

2.3.1 Logistic Regression

```
1 # 1. Logistic Regression dengan hyperparameter tuning
2 param_grid_lr = {
3     'C': [0.001, 0.01, 0.1, 1, 10, 100],
4     'penalty': ['l1', 'l2', 'elasticnet', None],
5     'solver': ['newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'],
6     'max_iter': [1000, 2000, 3000]
7 }
8
9 # Gunakan subset parameter yang kompatibel
10 param_grid_lr_subset = {
11     'C': [0.001, 0.01, 0.1, 1, 10, 100],
12     'penalty': ['l2', None],
13     'solver': ['newton-cg', 'lbfgs', 'sag'],
14     'max_iter': [2000]
15 }
16
17 log_reg = LogisticRegression(random_state=42)
18 grid_search_lr = GridSearchCV(log_reg, param_grid_lr_subset, cv=5, scoring='accuracy', n_jobs=-1)
19 grid_search_lr.fit(X_train_resampled, y_train_resampled)
20
21 print("Parameter terbaik untuk Logistic Regression:")
22 print(grid_search_lr.best_params_)
23
24 # Evaluasi model dengan parameter terbaik
25 log_reg_best = grid_search_lr.best_estimator_
26 results_lr = evaluate_model(log_reg_best, X_train_resampled, y_train_resampled, X_test, y_test, "Logistic Regression (Optimized)")
```

Gambar 2.22 Kode Evaluasi Model Logistic Regression

```
Parameter terbaik untuk Logistic Regression:
{'C': 1, 'max_iter': 2000, 'penalty': 'l2', 'solver': 'newton-cg'}
```

Logistic Regression (Optimized):
Cross-Validation Accuracy: 0.8602 ± 0.0337
Test Accuracy: 0.8704
ROC AUC: 0.9364
Log Loss: 0.3064

Confusion Matrix:
[[14 3]
 [4 33]]

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.82	0.80	17
1	0.92	0.89	0.90	37
accuracy			0.87	54
macro avg	0.85	0.86	0.85	54
weighted avg	0.87	0.87	0.87	54

Gambar 2.23 Ouput Evaluasi Model Logistic Regression

Model Logistic Regression dengan tuning hyperparameter menghasilkan akurasi 87% dan ROC AUC 0.9364, menunjukkan kemampuan yang baik dalam membedakan kelas "Placed" dan "Not Placed". Dengan precision 0.92 dan recall 0.89 pada kelas "Placed", model ini menunjukkan performa yang stabil dan dapat diandalkan, meskipun terdapat beberapa false positives dan false negatives. Log loss yang rendah (0.3064) menandakan prediksi probabilitas yang cukup baik.

2.3.2 K-Nearest Neighbors

```
1 # 2. K-Nearest Neighbors dengan hyperparameter tuning
2 param_grid_knn = {
3     'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
4     'weights': ['uniform', 'distance'],
5     'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
6     'p': [1, 2]
7 }
8
9 knn = KNeighborsClassifier()
10 grid_search_knn = GridSearchCV(knn, param_grid_knn, cv=5, scoring='accuracy', n_jobs=-1)
11 grid_search_knn.fit(X_train_resampled, y_train_resampled)
12
13 print("Parameter terbaik untuk K-NN:")
14 print(grid_search_knn.best_params_)
15
16 # Evaluasi model dengan parameter terbaik
17 knn_best = grid_search_knn.best_estimator_
18 results_knn = evaluate_model(knn_best, X_train_resampled, y_train_resampled, X_test, y_test, "K-Nearest Neighbors (Optimized)")
```

Gambar 2.24 Kode Evaluasi Model K-Nearest Neighbors

```
Parameter terbaik untuk K-NN:
{'algorithm': 'auto', 'n_neighbors': 5, 'p': 1, 'weights': 'distance'}

-----
K-Nearest Neighbors (Optimized):
Cross-Validation Accuracy: 0.8743 ± 0.0449
Test Accuracy: 0.8333
ROC AUC: 0.9173
Log Loss: 0.9484

Confusion Matrix:
[[13  4]
 [ 5 32]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.72	0.76	0.74	17
1	0.89	0.86	0.88	37
accuracy			0.83	54
macro avg	0.81	0.81	0.81	54
weighted avg	0.84	0.83	0.83	54

Gambar 2.25 Output Evaluasi Model K-Nearest Neighbors

Model K-Nearest Neighbors (K-NN) yang telah dioptimalkan menunjukkan performa yang cukup baik dengan akurasi cross-validation sebesar 87,43% dan akurasi pada data uji sebesar 83,33%. Parameter terbaik yang ditemukan menggunakan 5 tetangga terdekat, jarak Manhattan, bobot berbasis jarak, dan algoritma otomatis. Nilai ROC AUC mencapai 0,9173 yang menandakan kemampuan klasifikasi yang sangat baik, meskipun nilai log loss masih cukup tinggi di 0,9484. Berdasarkan confusion matrix dan classification report, model lebih akurat dalam mengklasifikasikan kelas mayoritas (kelas 1), namun masih bisa ditingkatkan untuk kelas minoritas (kelas 0).

2.3.3 Naive Bayes

```
1 # 3. Naive Bayes
2 nb = GaussianNB()
3 results_nb = evaluate_model(nb, X_train_resampled, y_train_resampled, X_test, y_test, "Naive Bayes")
```

Gambar 2.26 Kode Evaluasi Model

```
-----
Naive Bayes:
Cross-Validation Accuracy: 0.8235 ± 0.0747
Test Accuracy: 0.8333
ROC AUC: 0.8760
Log Loss: 1.2760

Confusion Matrix:
[[13  4]
 [ 5 32]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.72	0.76	0.74	17
1	0.89	0.86	0.88	37
accuracy			0.83	54
macro avg	0.81	0.81	0.81	54
weighted avg	0.84	0.83	0.83	54

Gambar 2.27 Output Evaluasi Model

Model Naive Bayes menunjukkan performa yang cukup kompetitif dengan akurasi cross-validation sebesar 82,35% dan akurasi pada data uji sebesar 83,33%, setara dengan model K-NN. Nilai ROC AUC yang mencapai 0,8760 menunjukkan kemampuan klasifikasi yang baik, meskipun nilai log loss sebesar 1,2760 menandakan prediksi probabilitasnya kurang presisi dibanding K-NN. Confusion matrix dan classification report menunjukkan distribusi prediksi yang sama dengan K-NN, dengan performa lebih unggul pada kelas mayoritas (kelas 1) dibandingkan kelas 0. Secara keseluruhan, meski lebih sederhana, Naive Bayes tetap menjadi model yang layak dipertimbangkan.

2.3.4 Decision Tree dengan hyperparameter tuning

```
1 # 4. Decision Tree dengan hyperparameter tuning
2 param_grid_dt = {
3     'max_depth': [None, 5, 10, 15, 20],
4     'min_samples_split': [2, 5, 10],
5     'min_samples_leaf': [1, 2, 4],
6     'criterion': ['gini', 'entropy', 'log_loss']
7 }
8
9 dt = DecisionTreeClassifier(random_state=42)
10 grid_search_dt = GridSearchCV(dt, param_grid_dt, cv=5, scoring='accuracy', n_jobs=-1)
11 grid_search_dt.fit(X_train_resampled, y_train_resampled)
12
13 print("Parameter terbaik untuk Decision Tree:")
14 print(grid_search_dt.best_params_)
15
16 # Evaluasi model dengan parameter terbaik
17 dt_best = grid_search_dt.best_estimator_
18 results_dt = evaluate_model(dt_best, X_train_resampled, y_train_resampled, X_test, y_test, "Decision Tree (Optimized)")
19
```

Gambar 2.28 Kode Evaluasi Model

```
Parameter terbaik untuk Decision Tree:
{'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}

-----
Decision Tree (Optimized):
Cross-Validation Accuracy: 0.7927 ± 0.0442
Test Accuracy: 0.7963
ROC AUC: 0.7401
Log Loss: 7.3422

Confusion Matrix:
[[10  7]
 [ 4 33]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.71	0.59	0.65	17
1	0.82	0.89	0.86	37
accuracy			0.80	54
macro avg	0.77	0.74	0.75	54
weighted avg	0.79	0.80	0.79	54

Gambar 2.29 Output Evaluasi Model

Model Decision Tree yang telah dioptimalkan menggunakan kriteria entropi tanpa batasan kedalaman pohon menunjukkan akurasi cross-validation sebesar 79,27% dan akurasi uji sebesar 79,63%, sedikit lebih rendah dibanding K-NN dan Naive Bayes. Meskipun mampu mengklasifikasikan kelas mayoritas (kelas 1) dengan baik (recall 0.89), performanya pada kelas minoritas (kelas 0) kurang optimal (recall 0.59). Nilai ROC AUC sebesar 0.7401 menandakan kemampuan pemisahan kelas yang sedang, sementara nilai log loss yang tinggi (7.3422) menunjukkan prediksi probabilitas yang tidak stabil. Secara keseluruhan, Decision Tree lebih rentan terhadap overfitting dan kurang akurat dibanding model lainnya.

2.4 Evaluasi Model

- Mengumpulkan Semua Hasil dan Bandingkan akurasi model

```
1 # Kumpulkan semua hasil
2 all_results = [results_lr, results_knn, results_nb, results_dt]
3 model_names = [result['name'] for result in all_results]
4 accuracies = [result['accuracy'] for result in all_results]
5 cv_accuracies = [result['cv_accuracy'] for result in all_results]
6 roc_aucs = [result['roc_auc'] for result in all_results]
7 log_losses = [result['log_loss'] for result in all_results]
8
9 # Bandingkan akurasi model
10 result_df = pd.DataFrame({
11     'Model': model_names,
12     'Test Accuracy': accuracies,
13     'CV Accuracy': cv_accuracies,
14     'ROC AUC': roc_aucs,
15     'Log Loss': log_losses
16 }).sort_values('Test Accuracy', ascending=False)
17
18 print("Perbandingan Kinerja Model:")
19 print(result_df)
```

Gambar 2.30 Kode Mengumpulkan Semua Hasil dan Bandingkan akurasi model

Perbandingan Kinerja Model:

	Model	Test Accuracy	CV Accuracy	ROC AUC	\
0	Logistic Regression (Optimized)	0.870370	0.860202	0.936407	
1	K-Nearest Neighbors (Optimized)	0.833333	0.874343	0.917329	
2	Naive Bayes	0.833333	0.823535	0.875994	
3	Decision Tree (Optimized)	0.796296	0.792727	0.740064	
	Log Loss				
0	0.306426				
1	0.948402				
2	1.275962				
3	7.342226				

Gambar 2.31 Output Mengumpulkan Semua Hasil dan Bandingkan akurasi model

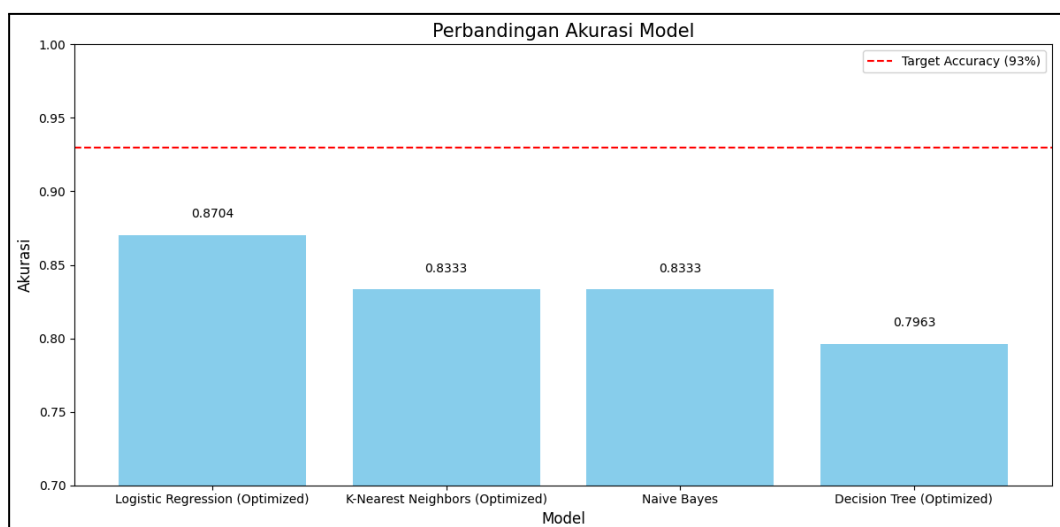
Tabel perbandingan menunjukkan bahwa Logistic Regression (Optimized) memiliki kinerja paling stabil dan unggul secara keseluruhan. Model ini meraih akurasi uji tertinggi (87,04%), ROC AUC terbaik (0,9364), serta log loss terendah (0,3064), yang menunjukkan prediksi probabilitasnya sangat akurat dan tidak

overconfident. K-Nearest Neighbors (K-NN) juga tampil baik dengan akurasi cross-validation tertinggi (87,43%) dan ROC AUC yang tinggi (0,9173), meskipun log loss-nya lebih tinggi dari Logistic Regression. Naive Bayes mempertahankan akurasi uji yang sama dengan K-NN (83,33%), namun memiliki log loss lebih tinggi, menandakan ketidakpastian dalam prediksi probabilitas. Sementara itu, Decision Tree, meskipun cukup baik dalam recall untuk kelas mayoritas, menunjukkan performa paling rendah dengan ROC AUC terendah (0,7401) dan log loss tertinggi (7,3422), mengindikasikan model ini kurang stabil dan cenderung overfit.

- **Visualisasi akurasi model**

```
1 # Visualisasi akurasi model
2 plt.figure(figsize=(12, 6))
3 bars = plt.bar(model_names, accuracies, color='skyblue')
4 plt.axhline(y=0.93, color='red', linestyle='--', label='Target Accuracy (93%)')
5 plt.title('Perbandingan Akurasi Model', fontsize=15)
6 plt.xlabel('Model', fontsize=12)
7 plt.ylabel('Akurasi', fontsize=12)
8 plt.ylim(0.7, 1.0)
9
10 # Tambahkan nilai akurasi di atas bar
11 for bar, acc in zip(bars, accuracies):
12     plt.text(bar.get_x() + bar.get_width()/2, acc + 0.01, f"{acc:.4f}",
13             ha='center', va='bottom', fontsize=10)
14
15 plt.legend()
16 plt.tight_layout()
17 plt.show()
```

Gambar 2.32 Kode Visualisasi akurasi model



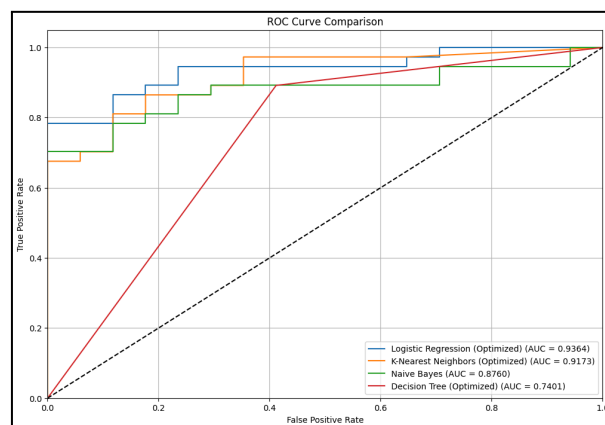
Gambar 2.33 Visualisasi akurasi model

Berdasarkan visualisasi bar chart di atas, terlihat bahwa tidak ada model yang mencapai target akurasi 93%, yang ditandai dengan garis merah putus-putus. Logistic Regression (Optimized) merupakan model dengan akurasi tertinggi sebesar 87,04%, diikuti oleh K-Nearest Neighbors (Optimized) dan Naive Bayes yang sama-sama mencapai 83,33%. Decision Tree (Optimized) memiliki akurasi paling rendah yaitu 79,63%. Grafik ini menunjukkan bahwa meskipun beberapa model mendekati target, masih diperlukan upaya peningkatan performa seperti pemilihan fitur yang lebih optimal atau tuning lebih lanjut untuk mencapai target akurasi.

- **Visualisasi ROC Curve untuk semua model**

```
1 # Visualisasi ROC Curve untuk semua model
2 plt.figure(figsize=(12, 8))
3
4 for result in all_results:
5     plt.plot(result['fpr'], result['tpr'], label=f"{result['name']} (AUC = {result['roc_auc']:.4f})")
6
7 plt.plot([0, 1], [0, 1], 'k--')
8 plt.xlim([0.0, 1.0])
9 plt.ylim([0.0, 1.05])
10 plt.xlabel('False Positive Rate')
11 plt.ylabel('True Positive Rate')
12 plt.title('ROC Curve Comparison')
13 plt.legend(loc='lower right')
14 plt.grid(True)
15 plt.show()
```

Gambar 2.34 Kode Visualisasi ROC Curve untuk semua model



Gambar 2.35 Visualisasi ROC Curve untuk semua model

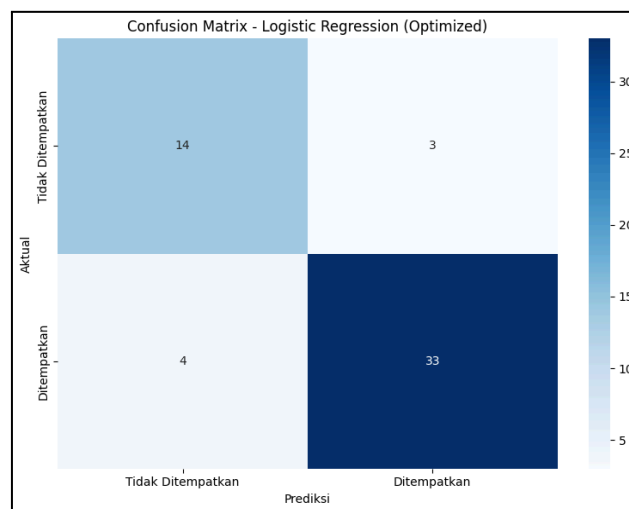
Grafik ROC Curve menunjukkan perbandingan performa empat model klasifikasi berdasarkan nilai AUC. Model Logistic Regression memberikan hasil terbaik dengan AUC sebesar 0.9364, diikuti oleh K-Nearest Neighbors (AUC = 0.9173) dan Naive Bayes (AUC = 0.8760) yang juga memiliki performa cukup baik. Sementara itu, Decision Tree memiliki performa paling

rendah dengan AUC sebesar 0.7401. Secara umum, semakin besar nilai AUC, semakin baik kemampuan model dalam membedakan antara kelas positif dan negatif.

- **Visualisasi confusion matrix untuk model terbaik**

```
1 # Visualisasi confusion matrix untuk model terbaik
2 best_model_idx = accuracies.index(max(accuracies))
3 best_model_name = model_names[best_model_idx]
4 best_model_pred = all_results[best_model_idx]['y_pred']
5
6 conf_matrix = confusion_matrix(y_test, best_model_pred)
7
8 plt.figure(figsize=(8, 6))
9 sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
10             xticklabels=['Tidak Ditempatkan', 'Ditempatkan'],
11             yticklabels=['Tidak Ditempatkan', 'Ditempatkan'])
12 plt.title(f'Confusion Matrix - {best_model_name}')
13 plt.xlabel('Prediksi')
14 plt.ylabel('Aktual')
15 plt.tight_layout()
16 plt.show()
```

Gambar 2.36 Kode Visualisasi confusion matrix untuk model terbaik



Gambar 2.37 Visualisasi confusion matrix untuk model terbaik

Model Logistic Regression yang telah dioptimalkan menunjukkan kinerja yang baik dengan jumlah prediksi benar sebanyak 47 dari total 54 data uji. Confusion matrix menunjukkan bahwa model mampu mengklasifikasikan mayoritas peserta dengan benar, terutama pada kategori "Ditempatkan". Kesalahan klasifikasi masih terjadi namun dalam jumlah kecil, sehingga model ini dapat dianggap cukup andal untuk prediksi status penempatan kerja.

2.5 Analisis Hasil

- Mengidentifikasi model terbaik

```
1 # Mengidentifikasi model terbaik
2 best_model = all_results[best_model_idx]['model']
3 best_model_name = all_results[best_model_idx]['name']
4 print(f"Model terbaik: {best_model_name} dengan akurasi {max(accuracies):.4f}")
```

Gambar 2.38 Mengidentifikasi model terbaik

```
Model terbaik: Logistic Regression (Optimized) dengan akurasi 0.8704
```

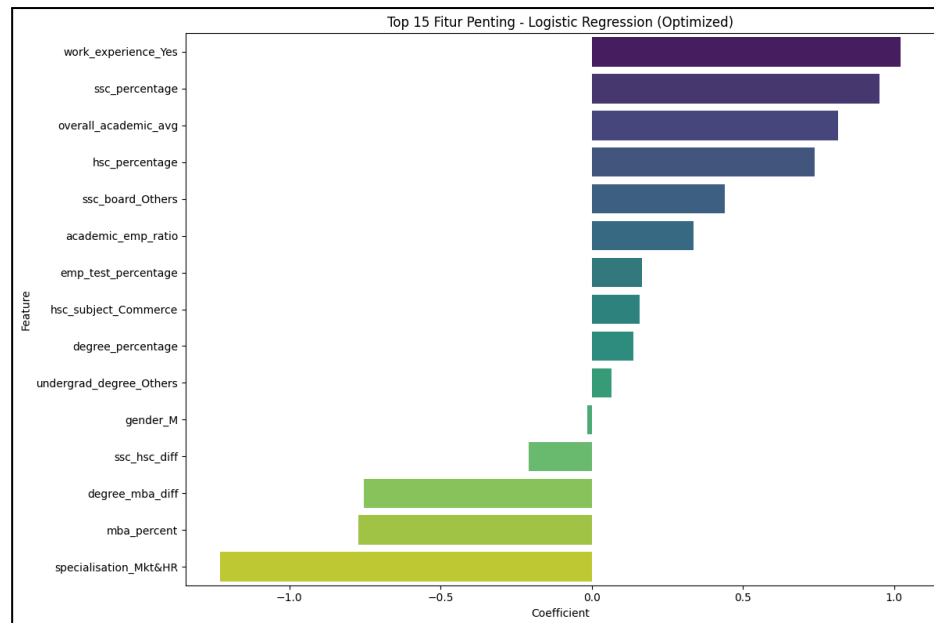
Gambar 2.39 Mengidentifikasi model terbaik

Model terbaik adalah Logistic Regression (Optimized) dengan akurasi 87,04%, ROC AUC tertinggi (0.9364), dan log loss terendah (0.3064), menunjukkan prediksi yang paling akurat dan konsisten dibanding model lain.

- Analisis fitur penting

```
1 # Analisis fitur penting
2 if hasattr(best_model, 'feature_importances_'):
3     # Untuk model berbasis tree (Decision Tree, Random Forest, Gradient Boosting)
4     importances = best_model.feature_importances_
5     feature_importance = pd.DataFrame({
6         'Feature': X.columns,
7         'Importance': importances
8     }).sort_values('Importance', ascending=False)
9
10    plt.figure(figsize=(12, 8))
11    sns.barplot(x='Importance', y='Feature', data=feature_importance.head(15), palette='viridis')
12    plt.title(f'Top 15 Fitur Penting - {best_model_name}')
13    plt.tight_layout()
14    plt.show()
15
16    print("Top 15 fitur paling penting:")
17    print(feature_importance.head(15))
18
19 elif hasattr(best_model, 'coef_'):
20     # Untuk model linier (Logistic Regression)
21     coefficients = best_model.coef_[0]
22     feature_importance = pd.DataFrame({
23         'Feature': X.columns,
24         'Coefficient': coefficients
25     }).sort_values('Coefficient', ascending=False)
26
27    plt.figure(figsize=(12, 8))
28    sns.barplot(x='Coefficient', y='Feature', data=feature_importance.head(15), palette='viridis')
29    plt.title(f'Top 15 Fitur Penting - {best_model_name}')
30    plt.tight_layout()
31    plt.show()
32
33    print("Top 15 fitur paling penting (berdasarkan koefisien):")
34    print(feature_importance.head(15))
35 else:
36    print("Model tidak memiliki atribut feature_importances_ atau coef_")
```

Gambar 2.40 Kode Analisis fitur penting



Gambar 2.41 Kode Analisis fitur penting

Berdasarkan visualisasi 15 fitur paling berpengaruh dalam model Logistic Regression, faktor yang paling meningkatkan peluang penempatan kerja adalah pengalaman kerja (*work_experience_Yes*), diikuti oleh nilai akademik seperti *ssc_percentage*, *overall_academic_avg*, dan *hsc_percentage*. Sementara itu, fitur seperti *specialisation_Mkt&HR*, *mba_percent*, dan perbedaan nilai antar jenjang pendidikan (*degree_mba_diff* dan *ssc_hsc_diff*) justru menunjukkan pengaruh negatif terhadap peluang penempatan. Hal ini menunjukkan bahwa kombinasi antara pengalaman kerja dan performa akademik yang konsisten menjadi penentu utama dalam keberhasilan penempatan, sedangkan latar belakang spesialisasi tertentu dan ketidakkonsistenan nilai cenderung menurunkan peluang tersebut.

Model ini menunjukkan bahwa faktor paling berpengaruh positif terhadap prediksi (misalnya kelulusan atau diterima kerja) adalah pengalaman kerja, nilai akademik seperti SSC dan rata-rata keseluruhan, serta rasio akademik dan tes kerja. Sebaliknya, faktor yang paling menurunkan peluang adalah spesialisasi Marketing & HR, nilai MBA tinggi, dan perbedaan besar antara nilai S1 dan S2. Beberapa fitur seperti jenis kelamin dan jurusan lain punya pengaruh sangat kecil terhadap hasil akhir.

BAB III

KESIMPULAN

Model terbaik dalam prediksi penempatan kerja adalah *Logistic Regression (Optimized)* dengan akurasi tinggi sebesar 87.04%, ROC AUC 93.64%, dan log loss rendah. Model ini menunjukkan kinerja sangat baik, khususnya dalam menyeimbangkan antara precision (91.67%) dan recall (89.19%) dengan F1-score sebesar 90.41%.

Proses preprocessing mencakup encoding data kategorikal, standardisasi numerik, pembuatan fitur baru (seperti selisih nilai dan rasio), serta penanganan kelas tidak seimbang dengan SMOTE. Berdasarkan analisis, nilai ujian penempatan dan pengalaman kerja adalah faktor paling menentukan dalam keberhasilan penempatan kerja. Selain itu, rekayasa fitur terbukti signifikan meningkatkan performa model. Disarankan untuk mahasiswa agar meningkatkan performa di ujian penempatan, mengusahakan pengalaman kerja sejak dini, memilih spesialisasi dengan peluang kerja tinggi, dan menjaga konsistensi nilai akademik.

BAB IV

DAFTAR PUSTAKA

- Bao, Y., Peng, Y., & Wu, C. (2023). Deep Learning-Based Job Placement in Distributed Machine Learning Clusters With Heterogeneous Workloads. *IEEE/ACM Transactions on Networking*, 31(2), 634–647. <https://doi.org/10.1109/TNET.2022.3202529>
- Danuri, D., & Jaroji, J. (2019). Elisitasi Kebutuhan Perangkat Lunak Rekrutmen Pegawai Dengan Pendekatan Soft System Methodology. *Just TI (Jurnal Sains Terapan Teknologi Informasi)*, 10(2), 42. <https://doi.org/10.46964/justti.v10i2.110>
- Metalita, E., Handiyani, H., Afriani, T., & Rayatin, L. (2021). Analisis Jenjang Karir dan Minat Menjadi Perawat Intensif. *Jurnal Keperawatan Silampari*, 5(1), 156–167. <https://doi.org/10.31539/jks.v5i1.2907>
- Raza, A. (2023). Job Placement Dataset [dataset]. <https://www.kaggle.com/datasets/ahsan81/job-placementdataset?resource=download>
- Umar, R., Fadlil, A., & Yuminah, Y. (2018). Sistem Pendukung Keputusan dengan Metode AHP untuk Penilaian Kompetensi Soft Skill Karyawan. *Khazanah Informatika : Jurnal Ilmu Komputer Dan Informatika*, 4(1), 27–34. <https://doi.org/10.23917/khif.v4i1.5978>